

Experiment - 4

Student Name: Harsh

Branch: MCA

Semester: 3rd

Subject Name: Machine Learning Lab

UID: 24MCA20045

Section/Group: 1 (B)

Date of Performance: 04/10/2025

Subject Code: 24CAP-702

Q. Implementation of Naïve Bayes and Decision Tree Algorithms

Aim/Overview of the practical:

- a) To implement the **Naïve Bayes algorithm** for solving a real-world classification problem using a suitable dataset.
- b) To implement a **Decision Tree classifier** using **Pandas** and **Graphviz** libraries, train the model, calculate **Gini Index** and **Information Gain**, and visualize the resulting tree.

Task to be Done:

1. For Naïve Bayes:

- Define a real-world problem suitable for Naïve Bayes (e.g., *Email Spam Detection*).
- Import and preprocess the dataset (text cleaning, feature extraction using CountVectorizer).
- Split the dataset into training and testing sets.
- Train the Naïve Bayes model (using MultinomialNB).
- Evaluate performance using accuracy score and classification report.
- Discuss the observed outcomes.

2. For Decision Tree:

- Load a suitable dataset (e.g., *Iris Dataset* or *Play Tennis Dataset*).
- Preprocess the data and select features and target variables.
- Train a Decision Tree classifier using DecisionTreeClassifier from sklearn.
- Compute **Gini Index** and **Information Gain** for feature selection.
- Visualize the tree using **Graphviz**.
- Analyze the decision rules formed by the model.

Code for Experiment

Part A – For Naive Bayes:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report

url = "https://raw.githubusercontent.com/justmarkham/pycon-2016-tutorial/master/data/sms.tsv"
df = pd.read_csv(url, sep='\t', names=['label', 'text'])

X = df['text']
y = df['label']

vectorizer = CountVectorizer()
X_vectorized = vectorizer.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_vectorized, y, test_size=0.3, random_state=42
)

model = MultinomialNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

print("Output 1:\n")
print(f"Accuracy: {accuracy:.2f}")
print("\nClassification Report:\n")
print(report)
```

Part B – For Decision Tree:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier

data = {
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain', 'Overcast', 'Sunny', 'Sunny', 'Rain'],
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool', 'Cool', 'Mild', 'Cool', 'Mild'],
    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal', 'High', 'High', 'Normal', 'Normal'],
    'Windy': [False, True, False, False, False, True, True, False, False, True],
    'Play': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No', 'Yes', 'Yes']
}

df = pd.DataFrame(data)

X = pd.get_dummies(df.drop('Play', axis=1))
y = df['Play']

model = DecisionTreeClassifier()
model.fit(X, y)

def gini_index(groups, classes):
    n_instances = float(sum([len(group) for group in groups]))
    gini = 0.0
    for group in groups:
        size = float(len(group))
        if size == 0:
            continue
        score = 0.0
        for class_val in classes:
            proportion = list(group).count(class_val) / size
            score += proportion ** 2
        gini += (1 - score) * (size / n_instances)
    return gini

group1 = df[df['Outlook'] == 'Sunny']['Play']
group2 = df[df['Outlook'] == 'Overcast']['Play']
group3 = df[df['Outlook'] == 'Rain']['Play']

groups = [group1, group2, group3]
classes = ['Yes', 'No']

gi = gini_index(groups, classes)
print(f"Gini Index for 'Outlook': {gi:.3f}")

from sklearn.tree import export_text
tree_rules = export_text(model, feature_names=list(X.columns))
print(tree_rules)
```

Output :-

Part A.

Output 1:

Accuracy: 0.99

Classification Report:

	precision	recall	f1-score	support
ham	0.99	0.99	0.99	1448
spam	0.93	0.96	0.95	224
accuracy			0.99	1672
macro avg	0.96	0.97	0.97	1672
weighted avg	0.99	0.99	0.99	1672

Part B.

```
Gini Index for 'Outlook': 0.300
|--- Outlook_Sunny <= 0.50
|   |--- Humidity_Normal <= 0.50
|   |   |--- class: Yes
|   |--- Humidity_Normal > 0.50
|   |   |--- Temperature_Cool <= 0.50
|   |   |   |--- class: Yes
|   |   |--- Temperature_Cool > 0.50
|   |   |   |--- Windy <= 0.50
|   |   |   |   |--- class: Yes
|   |   |   |--- Windy > 0.50
|   |   |   |   |--- class: No
|--- Outlook_Sunny > 0.50
|   |--- Humidity_Normal <= 0.50
|   |   |--- class: No
|   |--- Humidity_Normal > 0.50
|   |   |--- class: Yes
```



Learning Outcomes (What I Have Learnt):

- Apply **Naïve Bayes algorithm** to solve classification problems efficiently, especially in **text or categorical data** scenarios.
- Understand the working of **probabilistic classifiers** and how they compute class probabilities based on feature likelihood.
- Implement **Decision Tree classifiers** and interpret how decisions are made using **splits, Gini Index, and Information Gain**.
- Visualize and analyze a **Decision Tree model** to understand model interpretability and feature importance.
- Gain practical experience in **model training, evaluation, and visualization** using **Pandas, Scikit-learn, and Graphviz** in **Jupyter Notebook**.